

COMPLETE SYLLABUS

AI & ML Engineering

Complete Syllabus

16 modules, 75 skill sprints, and a project-first path from Linux shell and Python fundamentals to enterprise LLM platforms, MLOps, LLMOps, RAG, agents, multimodal AI and product-ready delivery.

16

Modules

75

Skill Sprints

16

Projects

1

Enterprise path

TABLE OF CONTENTS

Foundation to Core ML

Start with the engineering base, then move into the math, data, and classical ML skills that make the advanced AI chapters easier to learn.

Chapters 01-08

Engineering fundamentals, Python, math, data work, Scikit-learn, PyTorch, and computer vision.

Each chapter ends with a build milestone so the learning path stays practical instead of becoming theory-only.



CHAPTER 01

Engineering Bedrock: Git, Linux & HTTP

Build milestone: Publishable CLI Utility



CHAPTER 02

Python for AI Engineering

Build milestone: Data Pipeline CLI



CHAPTER 03

Math for Machine Learning

Build milestone: From-Scratch Regression Suite



CHAPTER 04

Scientific Python: NumPy, Pandas & Visualization

Build milestone: Exploratory Analytics Report



CHAPTER 05

SQL & Data Engineering for ML

Build milestone: Production Feature Pipeline



CHAPTER 06

Classical Machine Learning with Scikit-learn

Build milestone: Churn & Forecast Duo



CHAPTER 07

Deep Learning Foundations with PyTorch (+ Framework Landscape)

Build milestone: CIFAR-10 from Scratch



CHAPTER 08

Computer Vision & CNNs

Build milestone: Real-Time Detector Service

TABLE OF CONTENTS

Advanced AI to Enterprise Delivery

Continue into language models, agents, production ML, platform engineering, evaluation, observability, and the final capstone path.

Chapters 09-16

NLP, reinforcement learning, LLMs, RAG, agents, MLOps, LLMOps, multimodal AI, and enterprise delivery.

Use this second half as the enterprise-readiness track: every chapter connects modeling skill with production thinking.



CHAPTER 09

NLP & Sequence Models

Build milestone: Domain Sentiment API



CHAPTER 10

Reinforcement Learning

Build milestone: Bandit-Routed Prompt Optimizer



CHAPTER 11

Transformers & Large Language Models

Build milestone: Mini-GPT + Domain Assistant (pair)



CHAPTER 12

Prompt Engineering, RAG & LLM Evaluation

Build milestone: Evaluated Knowledge Assistant



CHAPTER 13

Agentic AI: Agents, MCP & Multi-Agent Systems

Build milestone: Governed Research Agent



CHAPTER 14

MLOps: Shipping Models to Production

Build milestone: Automated ML Pipeline



CHAPTER 15

LLMOps, Observability & Platform Engineering

Build milestone: Enterprise LLM Gateway Platform



CHAPTER 16

Multimodal AI, Product Thinking & Career Finishing

Build milestone: Multimodal Document Analyst + Capstone Case Study

16 Modules · 75 Skill Sprints · Project-First

Coverage: Python · Math for ML · NumPy/Pandas · SQL · Scikit-learn & Classical ML · Deep Learning & PyTorch (+ TensorFlow/Keras, JAX, Lightning) · CNN/Computer Vision · NLP · Reinforcement Learning · Transformers & LLMs · RAG · Vector Databases · Agentic AI · MLOps · LLMOps & Platform · Multimodal & AI Product.

LEARNING CHAPTER

MODULE 1 — Engineering Bedrock: Git, Linux & HTTP

Duration: 2–3 weeks · **Prerequisite:** none

Skill Sprint 1.1 — Linux & the Command Line

- Filesystem navigation, permissions (chmod/chown), symlinks
- Process management: ps, top/htop, kill, signals, exit codes
- Text power tools: grep, sed, awk, cut, sort, uniq, pipes & redirection
- Environment variables, PATH, .bashrc, shell scripting basics (loops, conditionals)
- SSH keys, scp/rsync, tmux for long-running training jobs

Skill Sprint 1.2 — Git & GitHub for ML Work

- init/clone/add/commit/push/pull, the staging-area mental model
- Branching: feature branches, trunk-based; merge vs rebase; conflict resolution
- .gitignore for ML (data/, models/, .env, checkpoints); Git LFS; never commit secrets
- Pull requests, code review etiquette, conventional commits, tagging model versions
- GitHub Actions intro: run pytest on every push

Skill Sprint 1.3 — HTTP, REST & APIs

- Request anatomy: verbs, headers, body, query params; status code families (422 vs 500)
- JSON serialization, content types, pagination, idempotency
- Auth: API keys, Bearer tokens, OAuth2 conceptually
- curl & Postman fluency; rate limits and retries

 **Labs:** shell script that watches a training log and alerts on failure · simulate and resolve a merge conflict · consume a public REST API with retry-on-429 handling.

BUILD MILESTONE

Module Project — Publishable CLI Utility

Build and publish a small CLI tool to GitHub with branch-protected main, CI badge, and a README a stranger can run in 5 minutes.

 **Deliverables:** public repo · passing CI · tagged v1.0 release.

LEARNING CHAPTER

MODULE 2 — Python for AI Engineering

Duration: 4–6 weeks · **Prerequisite:** Module 1

Skill Sprint 2.1 — Core Language

- Variables, types, mutability; truthiness; f-strings
- Control flow; enumerate, zip, range patterns
- Functions: positional/keyword/default args, *args/**kwargs, closures, type hints
- Comprehensions (list/dict/set), generators & yield, lazy iteration for big data

Skill Sprint 2.2 — OOP & Pythonic Design

- Classes, __init__, instance vs class attributes; inheritance & composition
- Dunder methods (__repr__, __len__, __getitem__ — why PyTorch Datasets use them)
- Dataclasses & Pydantic models; ABCs and protocols; when NOT to use OOP

Skill Sprint 2.3 — Robust Code

- File I/O: text, CSV, JSON, pathlib; context managers
- Exceptions: try/except/else/finally, custom exceptions, fail-fast philosophy
- Logging module (levels, handlers, structured logs) — print() is not logging

- Concurrency: threading vs multiprocessing vs asyncio; GIL implications; async/await basics (needed for FastAPI & agents)

⚡ Skill Sprint 2.4 — Professional Tooling

- venv/conda/uv; requirements.txt vs pyproject.toml; dependency pinning
- pytest: fixtures, parametrize, coverage; testing numeric code with tolerances
- Linting & formatting: ruff/black; pre-commit hooks; mypy basics
- Project layout: src/ packages; notebooks for exploration only

🔧 **Labs:** refactor a messy notebook into a tested package · typed config loader with Pydantic · async scraper for 50 URLs vs sync timing.

🏗 BUILD MILESTONE

Module Project — Data Pipeline CLI

Ingest messy CSVs → clean → validate → report, with logging, tests, and typed config; runs from a fresh venv.

📦 **Deliverables:** repo with src/ layout · ≥80% test coverage on utilities · README + sample output report.

📖 LEARNING CHAPTER

MODULE 3 — Math for Machine Learning

Duration: 6–8 weeks (run in parallel with Module 4) · **Prerequisite:** none

⚡ Skill Sprint 3.1 — Linear Algebra

- Vectors: norms, dot product geometry, cosine similarity (the embedding workhorse)
- Matrices: multiplication as transformation, transpose, inverse, rank
- Special matrices: identity, diagonal, orthogonal; projections
- Eigenvalues/eigenvectors; SVD — and its payoff in PCA and LoRA intuition

⚡ Skill Sprint 3.2 — Calculus & Optimization

- Derivatives as sensitivity; partial derivatives; gradient as steepest ascent
- Chain rule — the soul of backpropagation; computational graphs by hand
- Gradient descent: batch/mini-batch/stochastic; learning-rate dynamics; momentum, RMSProp, Adam intuition
- Convexity, local vs global minima, saddle points; Lagrange idea at a glance

⚡ Skill Sprint 3.3 — Probability

- Random variables; Bernoulli, Binomial, Gaussian, Poisson, Exponential
- Conditional probability, Bayes' theorem with diagnostic examples
- Expectation, variance, covariance & correlation
- Maximum likelihood — why MSE and cross-entropy are MLE in disguise
- Entropy, cross-entropy, KL divergence (losses + RLHF's KL penalty + drift detection)

⚡ Skill Sprint 3.4 — Statistics

- Descriptive stats; sampling distributions, Central Limit Theorem
- Confidence intervals; hypothesis testing, p-values, multiple-comparison trap
- A/B testing mechanics: power, minimum detectable effect
- Bootstrap & permutation tests

🧪 **Labs:** gradient descent from scratch matching sklearn · eigenvectors of a covariance matrix → derive PCA
 · Bayes simulation with synthetic disease testing · bootstrap a metric confidence interval.

🏗 BUILD MILESTONE

Module Project — From-Scratch Regression Suite

Linear + logistic regression in pure NumPy with loss curves and a written derivation document.

📄 **Deliverables:** sklearn-parity test in CI (within tolerance) · derivation PDF/markdown · loss-curve plots.

📖 LEARNING CHAPTER

MODULE 4 — Scientific Python: NumPy, Pandas & Visualization

Duration: 4–5 weeks · **Prerequisite:** Module 2

⚡ Skill Sprint 4.1 — NumPy Deep Skill

- ndarray model: shape, dtype, strides; views vs copies
- Broadcasting rules; axis semantics; fancy indexing & boolean masks
- Vectorization patterns; einsum; seeding for reproducibility
- Numerical stability: float traps, log-sum-exp, np.isclose

⚡ Skill Sprint 4.2 — Pandas Mastery

- Series/DataFrame internals; loc vs iloc; index discipline
- merge/join (fan-out detection), groupby-agg-transform, pivot/melt
- Time-series ops: resample, rolling, shift; timezone handling
- Missing-data strategies; categorical dtype; memory optimization; method chaining
- When pandas runs out: chunking, Polars/DuckDB awareness

⚡ Skill Sprint 4.3 — Visualization & EDA

- Matplotlib anatomy (figure/axes), Seaborn statistical plots, Plotly interactivity
- The EDA loop: distributions → relationships → anomalies → hypotheses
- Chart honesty: scales, baselines, when a table beats a chart
- Automated profiling (ydata-profiling) and its limits

🧪 **Labs:** beat a loop with broadcasting (measure 100×) · 3-table join with a planted fan-out bug to find · full EDA answering 5 business questions.

🏗 BUILD MILESTONE

Module Project — Exploratory Analytics Report

Messy real dataset (Airbnb/NYC taxi) → cleaning log → feature ideas → 10+ honest charts → insights memo.

📁 **Deliverables:** reproducible notebook + script version · data-quality report · 1-page insights memo.

MODULE 5 — SQL & Data Engineering for ML

Duration: 4–6 weeks · Prerequisite: Module 4

⚡ Skill Sprint 5.1 — SQL Fluency

- SELECT/WHERE/ORDER/LIMIT; aggregates, GROUP BY/HAVING
- Joins: inner/left/full/anti; key cardinality and fan-out
- Window functions: ROW_NUMBER, RANK, LAG/LEAD, rolling sums
- CTEs, subqueries, EXPLAIN basics, when indexes help

⚡ Skill Sprint 5.2 — Pipelines & Orchestration

- Idempotency, incremental loads, backfills
- cron+make → Airflow DAGs; retries & SLAs; dbt models and tests
- Data contracts: schema + freshness + ownership

⚡ Skill Sprint 5.3 — ML-Specific Data Engineering

- Point-in-time correctness (anti-leakage); training/serving skew
- Feature-store concepts (Feast): offline vs online, TTLs
- Validation gates with Great Expectations; lineage; schema evolution

🔧 **Labs:** rewrite a pandas pipeline in DuckDB SQL · build leaky vs point-in-time-correct features and measure the fake gain · add a validation gate that fails on injected bad data.

🏗️ BUILD MILESTONE

Module Project — Production Feature Pipeline

One feature table feeding both sklearn training and a FastAPI inference path, with validation gates and reproducible snapshots.

📁 **Deliverables:** pipeline repo with dbt/SQL models · failing-gate demo recording or log · snapshot reproducible by ID.

MODULE 6 — Classical Machine Learning with Scikit-learn

Duration: 8–10 weeks · **Prerequisite:** Modules 3–5

Skill Sprint 6.1 — The Supervised Core

- Linear regression: assumptions, residual analysis, Ridge/Lasso/ElasticNet
- Logistic regression: log-odds, decision thresholds, class weights
- k-NN, Naive Bayes, SVMs (margin intuition, kernels)
- Decision trees: splits, impurity, overfitting; Random Forests: bagging, OOB, feature-importance caveats

Skill Sprint 6.2 — Gradient Boosting (the tabular king)

- Boosting vs bagging; XGBoost vs LightGBM vs CatBoost
- Key knobs: depth, learning rate, estimators, subsampling, early stopping
- Categorical handling; monotonic constraints; imbalanced data (weights, SMOTE caution, threshold tuning)

Skill Sprint 6.3 — Unsupervised Learning

- K-means (choosing k, limits), DBSCAN/HDBSCAN, hierarchical clustering
- PCA in practice; t-SNE/UMAP for visualization (and their lies)
- Anomaly detection: Isolation Forest, one-class SVM
- Cluster validation: silhouette, stability, business sanity checks

Skill Sprint 6.4 — Evaluation & Validation (most interview-tested unit)

- Train/val/test discipline; stratification; k-fold; time-series splits; grouped splits
- Classification: confusion matrix, precision/recall/F1, ROC-AUC vs PR-AUC, calibration & Brier score
- Regression: MAE vs RMSE vs MAPE; quantile loss
- Leakage taxonomy: target, temporal, group, preprocessing leakage; cost-sensitive metric selection

⚡ Skill Sprint 6.5 — Pipelines, Tuning & Explainability

- Pipeline & ColumnTransformer; custom transformers; fit/transform discipline; joblib serialization & version pinning
- Hyperparameter search: grid → random → Optuna (TPE, pruning); nested-CV honesty
- SHAP (global & local), permutation importance, partial dependence; stakeholder-safe explanation

⚡ Skill Sprint 6.6 — Time Series & Recommenders

- TS: stationarity, decomposition, lag/rolling features, backtesting, naive & seasonal-naive baselines, SARIMA awareness, gradient-boosted TS, Prophet caveats
- RecSys: implicit vs explicit feedback, collaborative filtering, matrix factorization (ALS), precision@k & NDCG, cold start, popularity bias

⚡ Skill Sprint 6.7 — Responsible ML, Fairness & Model Risk

- Bias sources: sampling, labeling, proxy features, historical bias, evaluation blind spots
- Fairness metrics: demographic parity, equalized odds, subgroup performance, calibration by segment
- Model risk documents: assumptions, intended use, limitations, monitoring plan, rollback criteria
- Privacy basics: anonymization limits, differential privacy awareness, secure feature handling

🧪 **Labs:** churn baseline → boosted → calibrated comparison · leakage audit on a provided leaky notebook · Optuna study with pruning · SHAP report for non-technical readers · sales backtest with proper TS splits · subgroup fairness audit with remediation notes.

🏗️ BUILD MILESTONE

Module Project — Churn & Forecast Duo

(a) Churn classifier: 5-model comparison, calibration curve, SHAP report, Streamlit demo. (b) Demand forecaster: backtesting framework beating naive and seasonal-naive baselines.

📦 **Deliverables:** two repos (or monorepo) · completed leakage checklists · metric comparison tables · live Streamlit demo link.

MODULE 7 — Deep Learning Foundations with PyTorch (+ Framework Landscape)

Duration: 6–8 weeks · Prerequisite: Modules 3, 6

⚡ Skill Sprint 7.1 — Neural Network Mechanics

- Perceptron → MLP; activations (ReLU family, sigmoid/tanh, GELU); universal approximation intuition
- Forward pass as matrix ops; loss surfaces; backpropagation step-by-step on paper
- Weight initialization (Xavier/He); vanishing/exploding gradients

⚡ Skill Sprint 7.2 — PyTorch Core

- Tensors, autograd, computational graph; `requires_grad`, `no_grad`, `detach`
- `nn.Module` anatomy; `Dataset/DataLoader` (`workers`, pinned memory, `collate_fn`)
- The canonical loop: `zero_grad` → forward → loss → backward → step
- Device management (CPU/GPU/MPS); mixed precision (`autocast/GradScaler`); gradient clipping & accumulation
- Saving/loading: `state_dict`, checkpoints with optimizer state; TorchScript/ONNX export

⚡ Skill Sprint 7.3 — Training Craft

- Overfit-one-batch sanity test; diagnosing under/overfitting from curves
- Regularization: dropout, weight decay, label smoothing, early stopping, augmentation-as-regularizer
- BatchNorm/LayerNorm (and train vs eval mode bugs)
- LR schedules: step, cosine, warmup, one-cycle; TensorBoard/W&B tracking
- Debugging playbook: NaN losses, dead ReLUs, shape errors, dataloader bottlenecks

⚡ Skill Sprint 7.4 — Framework Landscape

- PyTorch Lightning: organized code, less boilerplate, multi-GPU (DDP concept)
- TensorFlow/Keras: reading & porting Keras code, SavedModel, where TF survives in industry (TFLite/TF-Serving)
- JAX: `jit/grad/vmap` functional style — awareness level
- Hugging Face ecosystem as the de facto layer: transformers, datasets, accelerate
- Strategy: PyTorch as home base; read the others fluently

⚡ Skill Sprint 7.5 — Modern Deep Learning Architectures

- Autoencoders and VAEs: compression, anomaly detection, latent spaces
- GANs: generator/discriminator training, mode collapse, when GANs still matter
- Diffusion models: denoising objective, U-Net backbone, schedulers, guidance, text-to-image intuition
- Self-supervised learning: contrastive learning, masked modeling, representation reuse

🧪 **Labs:** overfit one batch deliberately · MLP→CNN on Fashion-MNIST with ablations (no BN, no dropout, bad init) · resume training from checkpoint · port a small Keras model to PyTorch · train a tiny autoencoder and compare reconstruction errors.

🏗 BUILD MILESTONE

Module Project — CIFAR-10 from Scratch

Custom CNN achieving $\geq 90\%$ validation accuracy with full experiment tracking, early stopping, and mixed precision.

📦 **Deliverables:** training repo with config-driven runs · TensorBoard/W&B screenshots · ONNX export + parity test · model card.

📖 LEARNING CHAPTER

MODULE 8 — Computer Vision & CNNs

Duration: 6–8 weeks · **Prerequisite:** Module 7

⚡ Skill Sprint 8.1 — Convolution Fundamentals

- Convolution as learned filters; kernels, stride, padding, receptive fields; pooling; parameter counting
- Feature hierarchies: edges → textures → parts → objects; visualizing filters & feature maps

⚡ Skill Sprint 8.2 — Architectures

- LeNet → AlexNet → VGG (depth); ResNet (skip connections — why they fixed deep training)
- EfficientNet (compound scaling); MobileNet (depthwise separable, edge deployment)

- Vision Transformers: patches as tokens, ViT vs CNN inductive bias; timm library

⚡ Skill Sprint 8.3 — Transfer Learning & Augmentation

- Frozen backbone → head training → progressive unfreezing; discriminative learning rates
- Albumentations pipelines; label-safe transforms; test-time augmentation; when augmentation hurts

⚡ Skill Sprint 8.4 — Detection & Segmentation

- Detection: bounding boxes, anchors vs anchor-free, IoU, NMS, mAP@50 vs mAP@50:95; YOLO family; DETR idea
- Segmentation: semantic vs instance; U-Net; Dice/IoU evaluation
- Annotation workflow (Label Studio); dataset-quality auditing; hard-negative mining

⚡ Skill Sprint 8.5 — CV in Production

- Latency/accuracy tradeoffs; quantization & pruning awareness; ONNX Runtime/TensorRT concept
- Video & webcam streams (OpenCV); drift in visual data (lighting, cameras, seasons)

⚡ Skill Sprint 8.6 — Document AI, OCR & Visual Retrieval

- OCR pipelines: layout detection, table extraction, reading order, confidence thresholds
- Document understanding: forms, invoices, reports, scanned PDFs, page-level citation requirements
- Visual search: CLIP embeddings, image-vector retrieval, duplicate and near-duplicate detection
- CV quality assurance: labeling drift, annotator agreement, visual regression tests

🧪 **Labs:** filter visualization · fine-tune 2 backbones → accuracy-vs-latency table · annotate 200 images and train detection · corrupt val images to simulate drift · build OCR extraction for 20 scanned pages and audit failures.

BUILD MILESTONE

Module Project — Real-Time Detector Service

Fine-tuned YOLO on a custom annotated dataset → FastAPI endpoint (<200 ms) → Gradio webcam demo.

 **Deliverables:** annotated dataset + labeling notes · mAP report · deployed demo · error analysis grouped by failure category.

LEARNING CHAPTER

MODULE 9 — NLP & Sequence Models

Duration: 6–8 weeks · **Prerequisite:** Module 7

Skill Sprint 9.1 — Text Representation & Embeddings

- Preprocessing reality check (what to normalize, what to keep); BoW & TF-IDF (still strong baselines)
- Word2Vec (CBOW/skip-gram, negative sampling), GloVe, FastText subwords
- Embeddings deep dive (the bridge to RAG): cosine vs dot vs Euclidean, normalization effects, dimensionality tradeoffs, sentence embeddings (SBERT idea), ANN indexes (HNSW/IVF) at concept level

Skill Sprint 9.2 — Sequence Models

- RNN mechanics & vanishing gradient; LSTM/GRU gates
- Seq2seq encoder-decoder; the bottleneck problem → attention as the fix
- Attention internals: Q/K/V, weights, masking — pre-transformer framing

Skill Sprint 9.3 — Tokenization

- BPE, WordPiece, SentencePiece; vocab-size tradeoffs
- Token-length cost math; multilingual & code tokenization quirks; OOV behavior

Skill Sprint 9.4 — Applied NLP with Hugging Face

- pipeline → AutoModel/AutoTokenizer; fine-tuning BERT-class models with Trainer

- Tasks: classification, NER, QA, summarization; macro vs micro F1; error-taxonomy practice
- spaCy for fast production NLP; HF Hub & Spaces publishing

 **Labs:** TF-IDF + logistic regression baseline first, then beat it with BERT and quantify the cost of the gap · semantic search over 10k docs with sentence embeddings + FAISS · tokenize the same text with 3 tokenizers and compare counts/costs.

BUILD MILESTONE

Module Project — Domain Sentiment API

Fine-tuned transformer vs LSTM vs TF-IDF baselines on a domain dataset (e.g., financial news), deployed to HF Spaces.

 **Deliverables:** macro-F1 + confusion matrix · 50-error taxonomy document · public Space demo · baseline-comparison table.

LEARNING CHAPTER

MODULE 10 — Reinforcement Learning

Duration: 5–6 weeks · **Prerequisite:** Modules 3, 7

Why: RL is the engine behind RLHF/DPO (LLM alignment), bandit-based routing/recommendation, and agentic decision-making.

Skill Sprint 10.1 — Foundations: Bandits & MDPs

- Multi-armed bandits: ϵ -greedy, UCB, Thompson sampling — real use: A/B testing & prompt/model routing
- MDP formalism: states, actions, rewards, transitions, discount γ
- Policies, value functions V/Q , Bellman equations; exploration vs exploitation

Skill Sprint 10.2 — Classical Solution Methods

- Dynamic programming: policy/value iteration on gridworlds
- Monte Carlo methods; TD learning; SARSA vs Q-learning (on- vs off-policy)
- Tabular Q-learning hands-on: FrozenLake/Taxi with Gymnasium

⚡ Skill Sprint 10.3 — Deep RL

- DQN: replay buffer, target networks, why naive deep Q diverges; Double/Dueling DQN ideas
- Policy gradients: REINFORCE, variance problem, baselines
- Actor-critic → A2C → PPO (clipped objective — the industry workhorse)
- Practical reality: reward hacking, sample inefficiency, seed sensitivity, when RL is the wrong tool

⚡ Skill Sprint 10.4 — RL for LLMs (the bridge)

- RLHF pipeline: SFT → reward model from preferences → PPO with KL penalty
- DPO/ORPO: preference optimization without an explicit RL loop — why it's replacing classic RLHF
- Reward hacking in LLMs (sycophancy, verbosity); RLAIIF awareness; bandits for prompt/model selection in production

⚡ Skill Sprint 10.5 — RL for Agents, Routing & Decision Systems

- Contextual bandits for prompt, model, retriever, and tool routing
- Offline RL awareness: logged-policy bias, counterfactual evaluation, why production RL is risky
- Reward design for agentic workflows: task success, cost, latency, safety, human override rate
- Simulation-first rollout: replay logs, shadow mode, guardrails before online learning

🧪 **Labs:** Thompson-sampling A/B simulator vs fixed split (measure regret) · tabular Q-learning on FrozenLake with learning curve · DQN on CartPole · PPO on LunarLander with Stable-Baselines3, then read SB3's clipped-loss source · tiny DPO fine-tune on preference pairs with TRL · replay-based router evaluation before live traffic.

🏗️ BUILD MILESTONE

Module Project — Bandit-Routed Prompt Optimizer

A production-flavored router that picks among 3 prompts/models per request via Thompson sampling, logging regret over time.

🧪 **Deliverables:** simulator + live-router code · regret curves · report quantifying lift vs the worst arm · short note on where this maps to A/B testing.

MODULE 11 — Transformers & Large Language Models

Duration: 6–8 weeks · **Prerequisite:** Module 9 (Module 10.4 helpful)

Skill Sprint 11.1 — Architecture from Scratch

- Scaled dot-product attention: Q/K/V, the $\sqrt{d}$ factor, causal masking
- Multi-head attention, FFN blocks, residuals + LayerNorm (pre vs post)
- Positional information: sinusoidal, learned, RoPE, ALiBi; context-length tradeoffs
- Decoder-only vs encoder-only vs encoder-decoder (GPT/BERT/T5); KV-cache memory math
- Scaling-laws intuition; emergent-ability debate awareness

Skill Sprint 11.2 — Working with LLMs

- Sampling: greedy, temperature, top-k, top-p, repetition penalties — effects on output quality
- Context-window management; system vs user roles; structured output (JSON mode, function calling)
- API ecosystems (OpenAI/Anthropic/open-weights); local inference: llama.cpp, GGUF, Ollama
- Token economics: cost modeling per feature, latency budgets, streaming

Skill Sprint 11.3 — Fine-tuning & Alignment

- Full FT vs PEFT; LoRA mechanics (the low-rank/SVD payoff): rank, alpha, target modules; QLoRA
- SFT data quality > quantity; chat templates; instruction-data curation
- Alignment (connects to 10.4): RLHF, DPO with TRL; evaluating tuned models (held-out evals, MMLU-style, human preference)
- Quantization: GPTQ/AWQ/GGUF; measuring INT4 vs FP16 quality drop on YOUR eval set
- Distillation concept; when fine-tuning is the wrong answer (often — try RAG/prompting first)

Skill Sprint 11.4 — Inference & Serving

- vLLM: PagedAttention, continuous batching; TGI awareness
- Throughput vs latency; speculative decoding concept; cost per 1k requests

⚡ Skill Sprint 11.5 — LLM Systems Internals

- KV cache sizing, prefill vs decode latency, batching tradeoffs, context-window cost curves
- Quantization choices: weight-only vs activation-aware, quality gates per task, hardware fit
- Long-context behavior: lost-in-the-middle, retrieval vs long context, prompt packing
- Serving patterns: streaming, cancellation, timeouts, backpressure, graceful degradation

🔧 **Labs:** implement single-head attention and match `nn.MultiheadAttention` within tolerance · character-level mini-GPT (Karpathy-style), watch loss vs sample quality · LoRA-tune a 7B on 1k domain examples and eval vs base · quantize and measure the drop · serve with vLLM and `load-test` · benchmark prefill/decode latency at 4 context lengths.

🏗️ BUILD MILESTONE

Module Project — Mini-GPT + Domain Assistant (pair)

(a) From-scratch GPT with attention visualizations and one ablation (remove positional encoding, document the damage). (b) QLoRA domain assistant (medical/legal/code Q&A) served on vLLM.

📁 **Deliverables:** mini-GPT repo with generated samples · ablation write-up · before/after eval table for the tuned model · vLLM serving config + load-test numbers.

📖 LEARNING CHAPTER

MODULE 12 — Prompt Engineering, RAG & LLM Evaluation

Duration: 4–6 weeks · **Prerequisite:** Module 11 (Skill Sprints 11.1–11.2)

⚡ Skill Sprint 12.1 — Prompt Engineering as Engineering

- Zero/few-shot; chain-of-thought and when NOT to expose reasoning; system/role design
- Structured outputs: JSON schemas, validation, retry-on-invalid loops
- Prompt versioning & regression testing (prompts are code); injection-aware prompt design

⚡ Skill Sprint 12.2 — RAG Pipeline, End to End

- Ingestion: parsing PDFs/HTML, cleaning, metadata extraction
- Chunking: fixed/recursive/semantic/structural; size & overlap experiments
- Embedding-model selection (MTEB awareness); vector stores: FAISS vs managed (pgvector/Chroma/cloud)
- Retrieval: dense, BM25, hybrid + reciprocal rank fusion; metadata filtering; query rewriting/expansion (HyDE idea)
- Reranking: cross-encoders, when the latency is worth it
- Generation: grounding instructions, citation formats, "I don't know" behavior; awareness of parent-document, GraphRAG, agentic RAG

⚡ Skill Sprint 12.3 — Vector Databases & Retrieval Infrastructure

- Vector index internals: flat search, HNSW, IVF, PQ, recall/latency/memory tradeoffs
- Stores to understand: FAISS, pgvector, Qdrant, Milvus, Weaviate, Pinecone, Chroma
- Production concerns: namespaces, metadata filters, ACL-aware retrieval, deletes, re-indexing, backups
- Embedding lifecycle: model migration, dimension changes, drift checks, dual-write re-embedding rollout
- Retrieval debugging: nearest-neighbor inspection, query rewriting logs, recall@k, source coverage gaps

⚡ Skill Sprint 12.4 — Advanced RAG Patterns

- Parent-child retrieval, multi-vector retrieval, contextual compression, query decomposition
- GraphRAG and knowledge graphs: entity extraction, relation edges, graph traversal with citations
- Multimodal RAG: images, tables, charts, layout chunks, OCR confidence in retrieval ranking
- Long-context RAG: when to retrieve, when to pack, and how to prevent context poisoning

⚡ Skill Sprint 12.5 — LLM & RAG Evaluation (the differentiator skill)

- Golden sets: 50+ Q/A with sources, including adversarial & unanswerable questions
- RAGAS metrics: faithfulness, answer relevance, context precision/recall — and their blind spots
- LLM-as-judge: judge prompts, biases (position/length/self-preference), calibration against human labels
- Regression harness in CI; cost & latency as first-class metrics; failure taxonomies

🧪 **Labs:** chunk-size ablation (3 sizes × hybrid on/off → metric table) · compare pgvector vs Qdrant/FAISS on recall and latency · add a reranker, measure delta vs latency cost · build the 50-question golden set · wire RAGAS into pytest so a regression fails CI.

BUILD MILESTONE

Module Project — Evaluated Knowledge Assistant

100+ page corpus → hybrid retrieval + reranker → cited answers → chat UI, with evaluation as a first-class feature.

 **Deliverables:** golden eval set · RAGAS score dashboard · CI quality gate (demonstrated failing on a planted regression) · documented failure cases.

LEARNING CHAPTER

MODULE 13 — Agentic AI: Agents, MCP & Multi-Agent Systems

Duration: 6–8 weeks · **Prerequisite:** Module 12

Skill Sprint 13.1 — Single-Agent Foundations

- Workflow vs agent: routing, chaining, parallelization vs autonomous loops — choose the simplest that works
- ReAct loop (reason→act→observe); Reflexion/self-critique; bounded iterations by construction
- Tool design: narrow Pydantic schemas, validation both directions, idempotency for writes, least privilege
- Memory: conversation state, summarization, vector memory, forgetting policies
- LangGraph: typed state, nodes/edges, conditional routing, checkpointing, human-interrupt points

Skill Sprint 13.2 — MCP & Tool Protocols

- MCP primitives: tools, resources, prompts; server lifecycle; stdio vs HTTP transports
- Building an MCP server with auth, scopes, allowlists; approval gates for sensitive actions
- Treating tool results as untrusted input; sandboxing; audit-logging every call

Skill Sprint 13.3 — Multi-Agent Systems

- When multi-agent actually helps (and the honest costs: latency × N, error compounding)
- Patterns: supervisor/worker, planner-executor-critic, debate; handoffs & message contracts
- Frameworks: LangGraph-native vs CrewAI vs AutoGen — try each once, commit to one

- Termination guarantees, conflict resolution, shared-state discipline

⚡ Skill Sprint 13.4 — Agent Reliability & Safety

- Trajectory evaluation: tool-call correctness, step efficiency, goal-completion rates
- Prompt-injection attack surface of agents; write-action approval; rollback for agent mistakes
- Tracing every step (LangSmith/Langfuse); cost ceilings per run; fallback model routing

⚡ Skill Sprint 13.5 — Production Agent Architecture

- Durable execution: checkpoints, resumable runs, retries, queues, idempotent tool calls
- State design: short-term state, long-term memory, task ledger, audit trail, user-visible history
- Agent-to-system integration: APIs, databases, files, MCP tools, RBAC, approval workflows
- Reliability patterns: planner/executor separation, deterministic validators, timeouts, compensating actions

🧪 **Labs:** calculator/search/file tools with full validation · human approval before any write action · injection test where a poisoned document tries to trigger a tool — and fails · trajectory-eval harness scoring 20 recorded runs · resumable agent run with checkpoint restore.

🏗 BUILD MILESTONE

Module Project — Governed Research Agent

ReAct agent with web search + vector memory + your own MCP tool server, human-in-the-loop approvals, and bounded steps.

📦 **Deliverables:** full traces for 10 sample runs · trajectory-eval report · injection-test results · \$/run dashboard.

📖 LEARNING CHAPTER

MODULE 14 — MLOps: Shipping Models to Production

Duration: 6–8 weeks · **Prerequisite:** Modules 6–7, 1

⚡ Skill Sprint 14.1 — Packaging & Serving

- FastAPI model service: Pydantic request/response schemas, validation, health/readiness endpoints, async where it matters
- Docker: multi-stage builds, slim images, non-root, env-based config; docker-compose for local stacks
- Batch vs realtime vs streaming scoring — a decision framework

⚡ Skill Sprint 14.2 — Experiment & Artifact Management

- MLflow: runs, params, metrics, artifacts, model registry with stages
- Reproducibility = code + data + config + environment versions
- DVC for data versioning; dataset snapshots tied to model versions

⚡ Skill Sprint 14.3 — CI/CD for ML

- GitHub Actions: lint/type/test → build → integration test → deploy gates
- Data validation in the pipeline (Great Expectations); model-quality gates (minimum metric vs champion)
- Canary/shadow deployment for models; rollback drills (actually practice one)

⚡ Skill Sprint 14.4 — Post-Deployment

- Monitoring: input drift (PSI/KL), prediction drift, delayed-label strategies, proxy metrics
- Retraining triggers & automation; champion/challenger; feedback-capture loops

⚡ Skill Sprint 14.5 — Feature Stores, Online Serving & Governance

- Offline vs online feature stores, point-in-time joins, TTLs, freshness SLAs, backfill safety
- Online inference architecture: feature lookup, model service, cache, timeout budgets, fallbacks
- Model governance: approval gates, model cards, risk tiers, reproducibility evidence, audit trails
- Shadow, canary, blue-green, A/B, rollback, and champion/challenger release patterns

🧪 **Labs:** containerized FastAPI service with health checks + tests · MLflow-track 10 runs and promote one through registry stages · CI that blocks deploy on a deliberately bad model · drift detector that flags a shifted batch · online feature lookup with stale-feature fallback.

BUILD MILESTONE

Module Project — Automated ML Pipeline

DVC-versioned data → training → MLflow registry → Docker → FastAPI → CI with data & model quality gates → drift monitoring.

 **Deliverables:** end-to-end repo · CI run showing a blocked bad deploy · drift-alert demo · rollback executed once on purpose and documented.

LEARNING CHAPTER

MODULE 15 — LLMOps, Observability & Platform Engineering

Duration: 8–10 weeks · **Prerequisite:** Modules 12–14

Skill Sprint 15.1 — LLM Gateway Patterns

- Multi-provider routing (LiteLLM-style); fallbacks & retries with jitter; circuit breakers
- Semantic caching (embedding-similarity hits — the biggest cost lever); rate limiting; per-team auth & cost attribution
- Prompt management: versioned prompts, environments, A/B tests with statistical guardrails; bandit routing (← Module 10)

Skill Sprint 15.2 — Observability for AI Systems

- OpenTelemetry traces across API → retriever → LLM → tools; correlation IDs everywhere
- Langfuse/LangSmith: per-step latency & token cost, run tagging, sampling at volume
- RED metrics + AI-specific: tokens/request, cost/route, quality-score trends, cache hit rate
- SLOs & error budgets; burn-rate alerts; dashboards answering "up? good? what cost?"

Skill Sprint 15.3 — Guardrails, Security & Compliance

- Input rails: injection detection, PII redaction (Presidio), topic filters
- Output rails: secret/URL scanning, schema enforcement, safe fallbacks
- Secrets management (Vault/cloud), workload identity, key rotation

- OWASP LLM Top 10 mapped to your stack; red-team suites (Garak) in CI; audit trails; EU AI Act risk-tier awareness

⚡ Skill Sprint 15.4 — Cloud & Kubernetes for AI

- One cloud deeply: managed training, managed/serverless endpoints, model registry, IAM, cost tagging & budget alarms; Terraform for all of it
- K8s: Deployments/Services/Ingress, probes (and the model-load liveness trap), HPA on queue depth, GPU node pools & taints
- Async inference with queue workers (Celery/Ray); KServe/Ray Serve; blue-green & canary with auto-rollback
- FinOps for AI: \$/1k requests, GPU utilization, spot + checkpoint training

⚡ Skill Sprint 15.5 — Incident Engineering

- Runbooks per failure class: provider outage, retrieval degraded, cost spike, quality regression
- Rollback rehearsals; blameless postmortems; on-call design for AI systems

⚡ Skill Sprint 15.6 — LLM Release, Evaluation & Governance

- Prompt/model/retriever release gates: offline eval, canary eval, human review, rollback criteria
- Quality dimensions: correctness, groundedness, toxicity, privacy, refusal quality, tool-call accuracy
- Dataset governance: golden set ownership, adversarial cases, freshness, red-team scenario tracking
- Compliance posture: data retention, audit logs, consent, PII handling, model/vendor risk register

⚡ Skill Sprint 15.7 — LLM Cost, Capacity & Performance Engineering

- Capacity planning: token throughput, queue depth, concurrency, context-length mix, peak traffic
- Cost controls: semantic cache, response cache, model routing, prompt compression, max-token budgets
- Performance controls: streaming UX, speculative decoding awareness, async jobs, cancellation, backpressure
- Business telemetry: cost per workflow, quality per dollar, latency SLO, cache hit rate, route profitability

🧪 **Labs:** minimal LiteLLM gateway with fallback + semantic cache, measure cost savings on a replay log · trace a full RAG request end-to-end in Langfuse · run Garak against your own endpoint and fix top findings · kill the primary provider during a load test, watch fallback + alert fire · canary a prompt change with auto-rollback on planted regression · build a cost-per-workflow dashboard.

BUILD MILESTONE

Module Project — Enterprise LLM Gateway Platform

Multi-provider router + per-team auth & cost attribution + semantic cache + guardrails + versioned prompts + eval-gated CI + K8s autoscaling.

 **Deliverables:** gateway repo with Terraform · Grafana SLO board screenshots · chaos-drill recording/notes (provider kill) · three written runbooks.

LEARNING CHAPTER

MODULE 16 — Multimodal AI, Product Thinking & Career Finishing

Duration: 3–4 weeks core + ongoing · **Prerequisite:** Module 12

Skill Sprint 16.1 — Multimodal AI

- VLM prompting for images & documents; OCR strategy (Tesseract vs cloud vs VLM-as-OCR); layout-aware extraction
- CLIP-style image embeddings & multimodal retrieval
- Speech loop: VAD → Whisper STT → LLM → TTS; latency engineering for voice agents
- Evaluation: labeled extraction sets, WER/CER, human spot-audits

Skill Sprint 16.2 — AI Product Thinking

- Problem framing: user, baseline-without-AI, success metric in money/time, kill criteria
- Cost–latency–quality triangle; build/buy/fine-tune memos; regex-beats-LLM honesty
- Feedback loops: corrections → growing eval sets → retraining/refresh triggers; human escalation design

Skill Sprint 16.3 — Portfolio & Interviews

- Repo standard: README, ADRs, model card, risk register, ≤3-minute demo video
- ML system-design interviews: clarify → metrics → data → model → serving → monitoring; 10 drilled scenarios
- STAR narratives from your own projects; a maintained mistakes.md as interview ammunition

⚡ Skill Sprint 16.4 — Enterprise Capstone Integration

- Reference architecture: data pipeline → model/RAG/agent service → gateway → observability → governance
- System design artifacts: C4 diagram, sequence diagram, threat model, cost model, rollback runbook
- Product readiness: user journey, acceptance criteria, evaluation dashboard, incident-response plan
- Executive narrative: business problem, measurable value, risks, controls, and next iteration

🏗 BUILD MILESTONE

Module Project — Multimodal Document Analyst + Capstone Case Study

(a) Ingest PDFs with charts/images, extract text + layout, answer questions with page-level citations, optional audio briefing. (b) A polished case study of your strongest end-to-end system.

📎 **Deliverables:** extraction accuracy measured on 50 labeled fields · cited-answer demo · one-page exec summary + architecture diagram + metrics table + risk register · cost model · release checklist.

📖 LEARNING CHAPTER

Syllabus-Wide Rules

1. **Project-or-it-didn't-happen** — a module closes when its project repo passes the stranger-run test (someone else executes your README successfully).
2. **One tool per category** — PyTorch, one cloud, one orchestrator, one observability stack. Depth over logos.
3. **Gate before advancing** — the previous module's project deliverables must exist, and one older project must still run from a clean machine.
4. **mistakes.md** — one running failure catalog across all modules.
5. **Weekly rhythm** — 50% learn / 35% build / 15% document.

16 Modules · 75 Skill Sprints · 16 Projects · From Linux shell to enterprise LLM platform.

